CMPE 273 – DISTRIBUTED SYSTEMS

Week 7:

Naming and Service Discovery in Distributed Systems

Learning Objectives

By the end of this session you should be able to:

- Explain why service discovery is necessary in distributed systems
- Understand naming systems and service registries
- Compare client-side vs server-side discovery
- Understand DNS-based discovery, Consul, Eureka, and Kubernetes Service
- Implement a simple service registry and discovery mechanism
- Analyze how discovery interacts with load balancing and resilience

Opening Thought Experiment

You deploy **50 microservices** in the cloud.

Service A needs to call Service B.
But Service B:

- runs in containers
- auto-scales
- may restart anytime
- may move across nodes

How does Service A know **where Service B is running?**

Possible Answers

- hardcoded IP
- config file
- DNS
- registry

Problems

In distributed systems:

- IP addresses are ephemeral
- instances change dynamically
- services scale up/down

Therefore, we need:
Naming + Service Discovery

What is Naming?

Naming maps a **logical name → network location**
Example: payment-service → 10.1.4.23:8080

Examples in real life:

System	Example
DNS	google.com
Kubernetes	user-service.default.svc.cluster.local
Consul	payments.service.consul

What is Service Discovery?

Service discovery answers:
"Where is the service instance right now?"
Instead of fixed addresses.
Example:
Order Service asks registry:
"Where is Payment Service?"

Registry returns:

10.1.4.23:8080
10.1.5.10:8080
10.1.6.19:8080

Discovery is the **hidden backbone of the internet.**

What is Service Discovery?

Service discovery registers and maintains a record of all your services in a *service catalog*. This service catalog acts as a single source of truth that allows your services to query and communicate with each other.

Service discovery answers:
"Where is the service instance right now?"
Instead of fixed addresses.
Example:
Order Service asks registry:
"Where is Payment Service?"

Registry returns:

10.1.4.23:8080
10.1.5.10:8080
10.1.6.19:8080

https://developer.hashicorp.com/consul/docs/use-case/service-discovery

Why do we need Service Discovery?

Traditional Architecture:

App Server → Database
IP rarely changes.

Microservices Architecture:

Service A
↓
Service B (5 instances)
↓
Service C (autoscaling)

Problems:

- containers restart
- instances scale
- nodes fail
- deployments shift traffic

Without discovery:

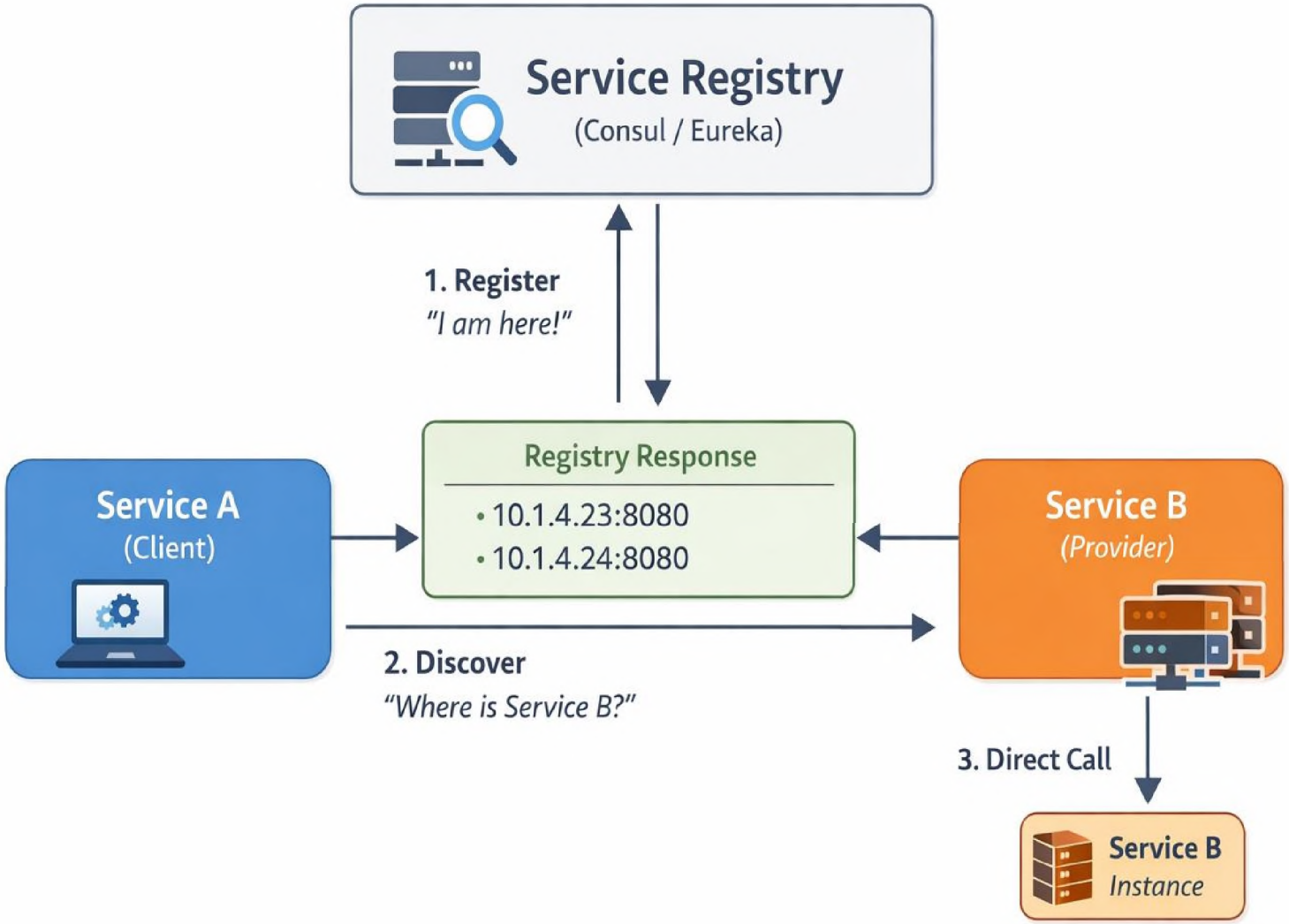
❌ hardcoded endpoints
❌ config chaos
❌ outages

With discovery:

✅ dynamic resolution
✅ load balancing
✅ resilience

https://developer.hashicorp.com/consul/docs/use-case/service-discovery

Service Registry Architecture



Step 1 – Registration
When a service starts:
Service B → Registry
"I am running at 10.1.4.23:8080"
The registry stores the service location.

Step 2 – Discovery
Service A needs Service B.
Instead of hardcoding an IP, it asks:
Service A → Registry
"Where is Service B?"
Registry returns available instances.

Step 3 – Service Call
Service A directly calls the instance.
Service A → Service B instance

Discovery Patterns

Pattern 1: Client-Side Discovery

- The service consumer queries the service catalog
- The service catalog retrieves and returns all access information
- The service consumer selects a healthy downstream service and makes requests directly to it

Example:

- Netflix Eureka
- Consul
- Zookeeper

Pros:

flexible

intelligent routing

Cons:

client complexity

Pattern 2: Server-Side Discovery

- The service consumer queries an intermediary (Consul)
- The intermediary queries the service catalog and routes requests to the available service instances.

Example:

- Kubernetes Service
- AWS ELB
- Nginx

Pros:

simple clients

infrastructure handles discovery

Cons:

extra hop

DNS based Discovery

Modern systems often use **DNS as service discovery**.
Example Kubernetes:
user-service.default.svc.cluster.local

- DNS resolves to service IP.
- DNS then load balances across pods.

Example flow:

```
Service A
   |
DNS Lookup
   |
user-service → 10.32.0.12
   |
Kubernetes proxy → Pod instances
```

Service Discovery vs Load Balancing

Service discovery and load balancing share a similarity in distributing requests to back end services, but differ in many important ways.

- Traditional load balancers are not designed for rapid registration and de-registration of services, nor are they designed for high-availability.
- By contrast, service discovery systems use multiple nodes that maintain the service registry state and a peer-to-peer state management system for increased resilience across any type of infrastructure.

For modern, cloud-based applications, service discovery is the preferred method for directing traffic to the right service provider due to its ability to scale and remain resilient, independent of infrastructure.

Class exercise: Build a simple Service registry

Simulate service discovery using this simple example…

https://github.com/ranjanr/ServiceRegistry

Try minikube and Hashicorp consul clients later…

Discussion Points:

- What happens if service crashes?
- How do we remove dead services?
- How do we load balance?
- How do we secure discovery?
- What happens across regions?

Quick Quiz

1. Why can't microservices use static IP addresses?
A. Containers move
B. Instances scale dynamically
C. Infrastructure changes
D. All of the above

2. What does service discovery solve?
A. Security
B. Dynamic service location
C. Code compilation
D. Storage

3. Kubernetes service discovery primarily uses:
A. Blockchain
B. DNS
C. FTP
D. RPC

Assignment : Service Discovery

Build a Microservice with Discovery

You must:

- Run **2 service instances**
- Register with registry
- Client discovers service
- Client calls random instance

Deliverables:

- GitHub repo
- architecture diagram
- demo video

Optional: Service Mesh Discovery

Istio / Linkerd

App → Sidecar Proxy → Service Mesh

Benefits:

- traffic routing
- observability
- security